ENSEA
Beyond Engineering

Lab Report

Magic Screen

An Etch A Sketch on HDMI with a Cyclone V FPGA

Fatima Zouhour

FPGA minor - TP report

Supervised by Antoine Tauvel

12 November 2026

Source code and demonstration video:
https://github.com/fatimazouhour/magic-screen-fpga

Summary

This report describes how I built a digital *magic screen* (an Etch A Sketch) on a DE10-Nano FPGA board with the TELECRAN shield. The two knobs of the shield move a stylus on a 720×480 picture sent on the HDMI output, the path of the stylus stays on the screen, and a push-button erases it. The work was done in five steps over three lab sessions: reading the encoders, producing the HDMI picture, moving a single pixel, memorising the drawing in a dual-port memory, and erasing it. For each step I explain how it works, how I implemented and checked it, and what I learned, including the problems I met and the decisions I had to rethink. The source code, this report and a demonstration video are available on GitHub: https://github.com/fatimazouhour/magic-screen-fpga.

Contents

Summary **1**

Introduction **2**

Step 1 – Reading the encoders **3**

Step 2 – Producing the HDMI picture **4**

Step 3 – Moving a pixel **5**

Step 4 – Memorising the drawing **6**

Step 5 – Erasing **6**

The final design **7**

Problems and solutions **10**

Conclusion **10**

Appendix A – User guide **12**

Appendix B – Lab 1 questions **12**

Appendix C – Code snippets **13**

Introduction

The project

Etch A Sketch : two knobs move a stylus horizontally and vertically, the stylus leaves a line behind it, and it toy erases the drawing. The goal of this project is to reproduce it with digital hardware: the left encoder of the TELECRAN shield moves the stylus horizontally, the right encoder vertically, the picture is displayed on the HDMI output of the board, and a push-button on the left encoder erases the screen.

Material and tools

The design runs on a Terasic DE10-Nano board (Intel Cyclone V FPGA `5CSEBA6U23I7`) with the TELECRAN shield, which carries the two rotary encoders with their push-buttons and ten LEDs. The HDMI output of the board goes to an HDMI-to-USB adapter, so the picture can be watched in VLC on the laptop (figure 1). I used Intel Quartus Prime Lite 24.1 to compile, program the board and look at the generated circuit, ModelSim for the simulations. The PLL (50 MHz → 27 MHz), the HDMI chip configuration (I2C), the dual-port memory `dpram.vhd` and the pin assignments were provided; the HDMI controller was developed in a previous TD session dedicated to the topic.



Figure 1: The test setup: DE10-Nano with the TELECRAN shield, HDMI-to-USB adapter, and VLC on the laptop.

Approach

- **simulate** in ModelSim when a block can be tested alone;
- **test on the board**, with the LEDs or the HDMI picture;
- **check the circuit** built by Quartus with the RTL Viewer;
- **save every working step** with Git before starting the next one, so that only one small change separates the current version from the last one that worked.

Organisation of the work

The project was done during three lab sessions. The first session was mostly about the encoders and their simulation, and a first HDMI picture; the second about testing the encoders on the board and the HDMI controller; the third about the moving pixel, the memory and the eraser. The report follows the five steps of the handout rather than the sessions.

Overall architecture

Everything I wrote runs on a single 27 MHz clock, the pixel clock of a 720×480 picture at 60 images per second. It is produced by a PLL from the 50 MHz oscillator of the board, and the `locked` output of the PLL is used as the reset of the whole design, so nothing starts before the clock is stable (Snippet 1). The two encoders give the position (x, y) of the stylus. A dual-port memory holds one bit per pixel: one port writes a '1' where the stylus is, the other port is read by the HDMI controller to decide whether each pixel is white or black. To erase, the writing port is given to a counter that writes '0' everywhere. The circuit generated by Quartus is shown in figure 4.

Decisions I rethought

Some choices changed during the project; they are explained in the steps, but here is the list:

- **Encoder filtering.** The first encoder version followed the rules of the handout directly and was disturbed by contact bounce on the board; I added a synchronizer and a 1 ms sampling.
- **One bit per pixel.** The memory is configured with 1 bit per pixel instead of 8: a pixel is either drawn or not, so 1 bit is enough and uses eight times less memory.
- **Screen limits.** Instead of changing the encoder (which counts from 0 to 1023, as asked), the memory is simply not written when the stylus is outside the 720×480 area.
- **Eraser.** I first thought of a separate component with button-edge detection and a state machine. Comparing the time to erase the memory (12.8 ms) with the duration of a press (around 100 ms) showed that a counter and a multiplexer, written directly in the top file, are enough.
- **Erase on reset.** I noticed that KEY0 did not erase the drawing, because a memory is not cleared by the reset; one flag now makes the counter erase the memory once after each reset.
- **Button synchronizer.** Random white dots after erasing showed that the erase button also had to be synchronized, like the encoders.
- **Provided files.** I kept the provided files (memory, PLL, constraints) unchanged.

Step 1 – Reading the encoders

How it works

A rotary encoder contains two switches, A and B, that open and close one after the other when the knob turns. Turning right, A changes before B; turning left, B changes before A. The handout gives the rules: the register is incremented on a rising edge of A while $B = 0$ or a falling edge of A while $B = 1$, and decremented on a rising edge of B while $A = 0$ or a falling edge of B while $A = 1$. An edge is found by keeping the previous value of a signal in a flip-flop and comparing it with the current one. One notch of the knob contains two edges of A, so it changes the count by two.

Implementation

The component `encoder` has a configurable width (`reg_size`, 10 bits here, so the value can be shown on the ten LEDs) and a sampling period (`div`). Each input first goes through two

flip-flops in series (`r_A_sync`, `r_B_sync`), because the switches are not synchronous with the clock. A counter (`r_div`) produces a one-cycle pulse `r_tick` every 27 000 cycles, i.e. every 1 ms (Snippet 2); the inputs are only examined on this pulse, which hides the bounce of the contacts. On each pulse the four rules are applied, `r_A` and `r_B` holding the previous values, and the 10-bit counter `r_cnt` goes up or down (Snippet 3).

Verification

The testbench `encoder_tb` uses `div => 1` so that the inputs are sampled at every cycle, applies one period to the right and one to the left, and checks the counter with `assert` (Snippet 4). The counter goes $0 \to 1 \to 2$ on the two edges of A, then back to 0 on the two edges of B (figure 2). On the board, the two values are shown on the LEDs and change by two at each notch (video `documentation/encoder-led-test.mp4`).

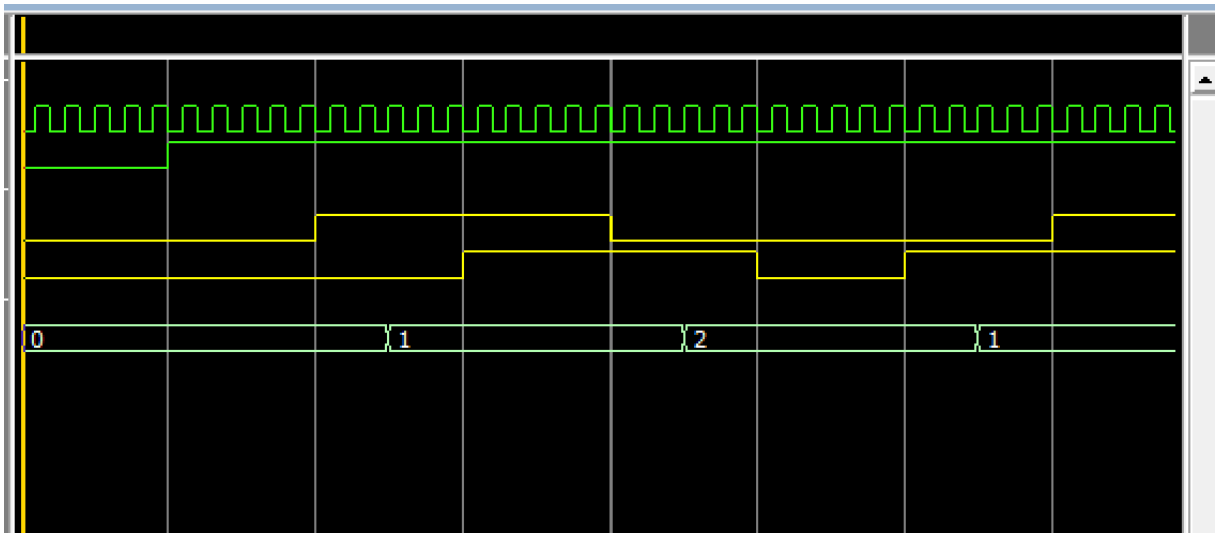


Figure 2: Simulation of the encoder (from top: clock, reset, A, B, counter).

How I approached it. My first version applied the rules directly. It was correct in simulation, but on the board the LEDs sometimes jumped by several counts for one notch: the contacts bounce, and the signals are not synchronous with the clock. The synchronizer and the 1 ms sampling solved it. Thanks to the generic `div`, the same code is fast to simulate and filtered on the board.

What I learned. An edge detector is a comparison between the previous and the current value of a signal. A simulation with perfect signals does not show the problems of real ones: inputs coming from outside must be synchronized and, for mechanical contacts, filtered.

Step 2 – Producing the HDMI picture

How it works

A screen receives its picture one pixel at a time, line after line, at the rhythm of the pixel clock, sixty times per second. Each line and each picture also contain an invisible part used by the screen to synchronise. The HDMI controller is made of two counters: one counts the clock cycles of a line (858: 61 for the synchronisation pulse, 58 and 18 for the two margins, 720 visible), the other counts the lines of a picture (525: 5 + 30 + 9 invisible, 480 visible) (Snippet 5). Comparisons on these counters produce the synchronisation pulses `hs` and `vs`, the signal `de`

which is '1' in the visible area, the coordinates of the pixel being sent, and its address $x + 720\,y$ (Snippet 6, Snippet 7).

Implementation and verification

I added the controller of the tutorial to the project and connected it to the 27 MHz clock and the reset of the PLL. To test it, the handout suggests driving the colour with the counters: x on bits 23 to 16 and y on bits 15 to 8 of `o_hdmi_tx_d` (Snippet 8). The result is figure 3.

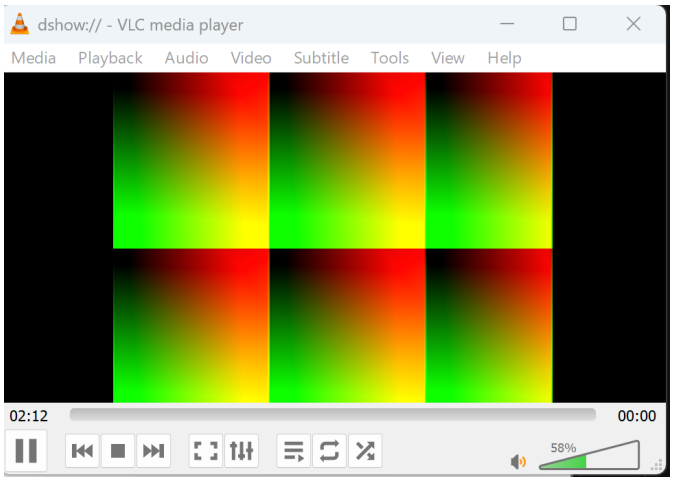


Figure 3: Colour test captured in VLC.

The part driven by x grows from left to right and is red, the part driven by y grows from top to bottom and is green, and there is no blue: bits 23 to 16 are the **red** component, bits 15 to 8 the **green** one and bits 7 to 0 the **blue** one. The picture shows three columns and two rows of squares because only 8 bits of each counter are used: the colour starts again every 256 pixels, and 720/256 and 480/256 give about three and two. This also shows that the whole 720×480 area is scanned.

What I learned. A video signal is only two counters and some comparisons: the picture exists only as a sequence of pixels sent at the right moment. A simple test pattern is a fast way to check the whole video chain on the hardware.

Step 3 – Moving a pixel

The left encoder now gives x and the right one y. As asked in the handout, the colour is white when the coordinates of the HDMI controller are equal to the values of the encoders, and black otherwise (Snippet 9): two comparators, an AND gate and a multiplexer. There is nothing to simulate; I tested it on the board. After a reset the counters are at 0, so the pixel starts in the top-left corner; above 719 or 479 it is outside the visible area.

How I approached it. At first I thought it did not work: VLC showed coloured bars, then a black screen. The bars only appear while KEY0 is held (there is no HDMI signal, so the adapter shows its own pattern), and a single pixel in a corner is very hard to see. After turning both knobs, the pixel appeared and followed them. This version is kept in comments in the final code.

Step 4 – Memorising the drawing

How it works

To keep the path of the stylus, every pixel must be remembered. The provided `dpram.vhd` is a *dual-port* memory: it has two independent ports, A and B, each with its own address, data, write enable and clock, and both can be used at the same time. As suggested by the handout, port A writes the pixels at the position of the encoders, and port B is read by the HDMI controller. The writing side and the display never wait for each other.

Implementation

One bit per pixel is enough (drawn or not), so the memory holds $720 \times 480 = 345\,600$ bits (Snippet 10). Its data ports are vectors; with one bit they are connected through bit 0 (`i_data_a(0)`, `o_q_b(0)`).

- **Writing (port A):** the address $x + 720\,y$ is computed from the encoders with a multiplier and an adder. Since the counters can reach 1023, the memory is only written when the stylus is inside the screen (`s_cursor_on_screen`). The same lines contain the multiplexer used later by the eraser (Snippet 11).
- **Reading (port B):** the address is the pixel address of the HDMI controller, which was left unconnected until now. The bit read gives the colour: white for '1', black for '0' (Snippet 12).
- **Timing:** the memory gives its data one clock cycle after the address; the HDMI controller also delays `de` by one flip-flop, so the data and `de` arrive together and the picture is not shifted.

Verification

On the board, the path of the stylus now stays on the screen (see the demonstration video). The compilation report confirms that the memory is built in the memory blocks of the FPGA (345 600 memory bits).

How I approached it. I first decided who writes in the memory, who reads it, and with which addresses. The read address already existed in the controller, so I only had to connect it. My first compilation failed only because I had written `dparam` instead of `dpram`. Then I replaced the moving-pixel process by one line that takes the colour from the memory.

What I learned. A frame-buffer separates what is drawn from what is displayed, and a dual-port memory lets both happen at the same time. A memory read takes one clock cycle, and the rest of the design must be aligned with it.

Step 5 – Erasing

How it works

The handout says that erasing means going through all the memory addresses to write a zero. Port B is busy with the display, so the erase goes through port A, which is normally used by the stylus. "Going through all the addresses" is a counter, and "giving port A either to the stylus or to the counter" is a multiplexer.

Implementation

- A counter `s_erase_addr` goes from 0 to 345 599 and starts again, all the time; one complete sweep takes 12.8 ms (Snippet 13).
- The select signal `s_erasing` drives the multiplexer of port A: when it is '1', port A takes the address of the counter and writes '0'; otherwise it takes the address of the stylus and writes '1' (Snippet 11).
- `s_erasing` is '1' while the left button is held (it gives '0' when pressed) or during the first sweep after a reset (flag `r_clear_done`), so that KEY0 also erases the screen.
- The button goes through a 2-flip-flop synchronizer (`r_pb_sync`), like the encoder inputs.

A press lasts around 100 ms, much longer than a sweep, so every address is written to '0' several times while the button is held.

Verification

On the board, holding the left button erases the drawing, and drawing works again when it is released; KEY0 also gives a blank screen (see the demonstration video).

How I approached it. I chose the simplest solution: a counter that runs all the time and a multiplexer that gives port A to the counter while the button is held. Then I noticed that KEY0 did not erase the drawing, because a memory is not cleared by the reset, and added `r_clear_done`. Finally, a few white dots sometimes remained after erasing, at random places. The button was used directly; it is asynchronous and bounces, and the select signal controls both the address and the data of port A through paths that do not have exactly the same delay. When the button changed just before a clock edge, the memory could take the address of the counter with the data '1' of the drawing mode, and wrote a white pixel wherever the counter was. With the synchronizer, the select signal only changes on a clock edge and the dots disappeared.

What I learned. Comparing orders of magnitude can make a design much simpler. When two functions share a resource, a multiplexer and a select signal are enough. A memory keeps its content after a reset. And every signal coming from outside must be synchronized; otherwise the errors are random and do not appear in simulation.

The final design

Generated circuit

Figure 4 is the circuit generated by Quartus from the final `telecran.vhd` (RTL Viewer). The blocks of the code can be recognised in it:

`Mult0, Add0`
: the write address $x + 720\,y$ (`10'h2d0` is 720);

`LessThan0, LessThan1, s_cursor_on_screen`
: the test $x < 720$ and $y < 480$ (`10'h1e0` is 480);

`s_wr_addr~, s_wr_addr`
: the two multiplexers of the write address (stylus, then eraser);

`r_pb_sync, s_erasing`
: the button synchronizer and the OR gate with inverted inputs that selects the erase mode;

`Add1, Equal0, s_erase_addr, r_clear_done`
: the erase counter, its end of sweep at `19'h545ff` (345 599), and the flag set after the first sweep;

`dpram:dpram0`
: the memory, whose port B never writes; the bit read is copied on the 24 colour bits.

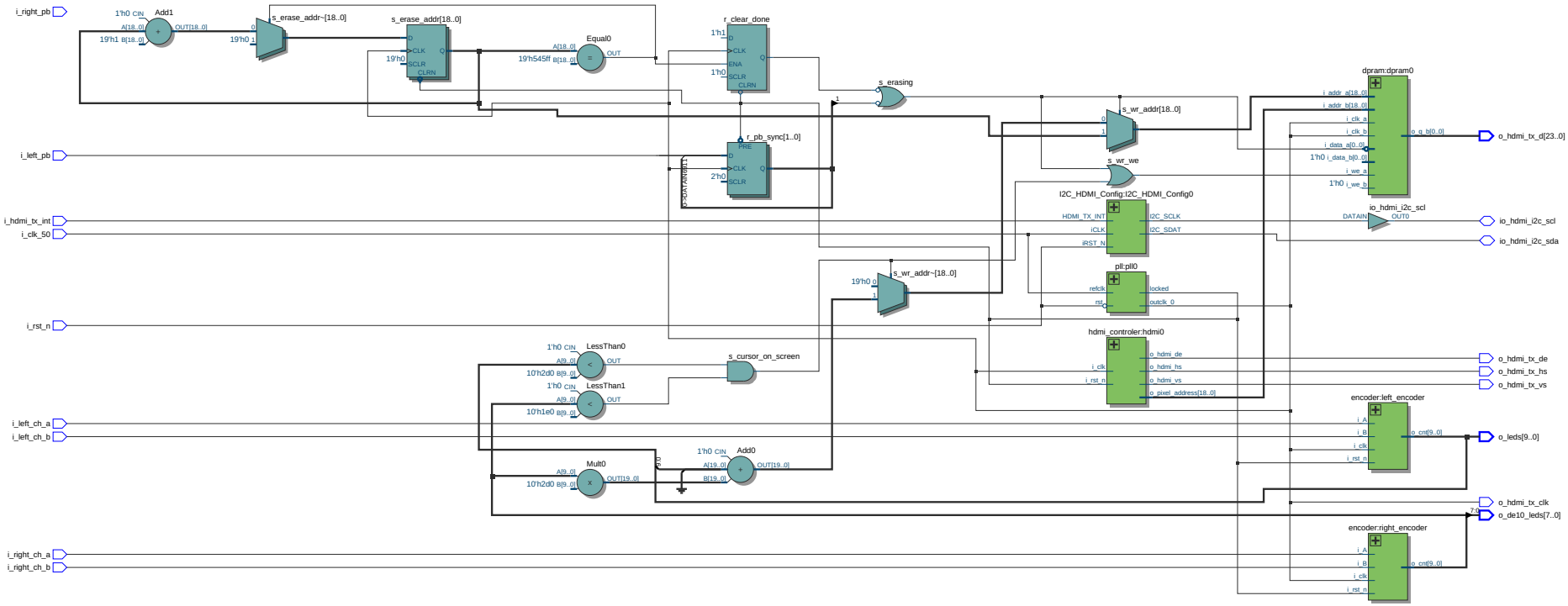


Figure 4: RTL view of the final design.

Resources and timing

According to the compilation report, the design uses 474 look-up tables, about 212 flip-flops, 345 600 memory bits (in 43 memory blocks), 2 DSP blocks for the two multiplications by 720 (write address and read address), one PLL and 57 pins. The memory is therefore really built in the memory blocks of the FPGA, not with flip-flops. The maximum frequency reported for the pixel clock is 104 MHz, well above the 27 MHz used.

The Timing Analyzer nevertheless shows errors. The provided constraint file declares the clocks on ports named `FPGA_CLK1_50`, which do not exist in the design (the clock port is `i_clk_50`), so Quartus assumes a default clock of 1 GHz and checks the design against impossible frequencies. This also made the compilation much slower once the memory was added. The design works on the board; the correct line would be `create_clock -period 20 [get_ports i_clk_50]`.

Demonstration

A video of the final design, drawing and erasing, is in the README of the GitHub repository: https://github.com/fatimazouhour/magic-screen-fpga. Some latency can be noticed in the video between turning a knob and seeing the line move. It is most likely due to the HDMI-to-USB converter and the capture software, which buffer the picture: in the design itself, a new position is taken into account within 1 ms and appears on the next picture, that is after at most about 17 ms.

Problems and solutions

The LEDs jumped by several counts for one notch.
Contact bounce and asynchronous inputs; solved with a synchronizer and a 1 ms sampling.

Coloured bars in VLC while KEY0 is held.
There is no HDMI signal during the reset, so the adapter shows its own test pattern; nothing to fix.

The moving pixel was not visible.
A single pixel in the corner, or a counter above 719 or 479; turning the knobs brought it back.

Compilation error on `dparam`.
A typing mistake: the entity is `dpram`.

Very slow compilation and timing errors.
The constraint file uses clock port names that do not exist; the real maximum frequency is well above what is needed.

A black band on the left of the picture.
VLC adds black bars around the picture; *Video* > *Aspect Ratio* removes them.

KEY0 did not erase the drawing.
A memory is not cleared by the reset; the counter now erases it once after each reset.

Random white dots after erasing.
The erase button was not synchronized; a 2-flip-flop synchronizer solved it.

Conclusion

The magic screen works on the board: the knobs draw, the drawing stays, and the left button or KEY0 erases it. The five steps of the handout were completed, and each one was designed

as blocks, described in VHDL, checked in simulation or on the board, and compared with the circuit generated by Quartus.

The most important things I learned are the following. Hardware is described, not programmed: a `when ... else` becomes a multiplexer, a comparison a comparator, a clocked `if` a register, and all the blocks work at the same time. Small steps, each saved when it works, make problems easy to locate. Simulation and tests on the board are complementary, because real signals bounce and are not synchronous with the clock; I needed a synchronizer twice. Memories separate the producer and the consumer of data, have a read latency, and keep their content after a reset. Finally, comparing orders of magnitude led to a much simpler eraser than my first idea.

With more time, I would limit the counters to the size of the screen, use the right button (for example to change the colour), store more bits per pixel to draw in colour, and correct the constraint file.

Appendix A – User guide

Compile and program. Open `telecran/telecran.qpf` in Quartus and run *Processing* > *Start Compilation*. Power the board, connect the **USB BLASTER II** port to the PC, open *Tools* > *Programmer*, click *Auto Detect*, select `5CSEBA6`, choose `output_files/telecran.sof` with *Change File*, tick *Program/Configure* and click *Start*.

Display. Connect the HDMI output of the board to the HDMI-to-USB adapter and the adapter to the PC. In VLC, open *Media* > *Open Capture Device*, select the adapter and click *Play*.

Controls. The left knob moves the stylus horizontally and the right knob vertically; holding the left knob's button erases the screen; KEY0 resets the design, erases the screen and puts the stylus back in the top-left corner. The LEDs show the position in binary.

Simulation. In ModelSim, change the directory to `encoder/` and type `do encoder.do`.

Appendix B – Lab 1 questions

Inverted LED. With `led0 <= pushl;` the LED is on by default and turns off when the button is pressed, because the push-button gives '0' when pressed. The correct code is `led0 <= not pushl;`.

Clock pin. `FPGA_CLK1_50` is connected to pin V11 (figure 5).

Blinking LED. The code is a flip-flop whose input is its own output inverted, so the LED changes at every clock edge, far too fast to be seen. A counter producing a one-cycle enable every 5 000 000 cycles makes it visible.

Meaning of `_n`. `i_rst_n` is active low: the reset is active when the signal is '0'.

Table 3-5 Pin Assignment of Clock Inputs

Signal Name	FPGA Pin No.	Description	I/O Standard
FPGA_CLK1_50	PIN_V11	50 MHz clock input	3.3V
FPGA_CLK2_50	PIN_Y13	50 MHz clock input	3.3V
FPGA_CLK3_50	PIN_E11	50 MHz clock input (share with FPGA_CLK1_50)	3.3V
HPS_CLK1_25	PIN_E20	25 MHz clock input	3.3V
HPS_CLK2_25	PIN_D20	25 MHz clock input	3.3V

Figure 5: Clock inputs of the DE10-Nano (user manual).

Appendix C – Code snippets

The snippets are copied from the project files, with the line numbers of the files. The complete sources are on GitHub.

Snippet 1: `telecran/telecran.vhd`, lines 91–98: PLL, its `locked` output is the reset

```
91      -- Frequency for HDMI is 27MHz generated by this PLL
92      pll0 : component pll
93          port map (
94              refclk => i_clk_50,
95              rst => not(i_rst_n),
96              outclk_0 => s_clk_27,
97              locked => s_rst_n
98          );
```

Snippet 2: `encoder/encoder.vhd`, lines 49–58: synchronizer and 1 ms tick

```
49              r_A_sync <= r_A_sync(0) & i_A;
50              r_B_sync <= r_B_sync(0) & i_B;
51
52              if r_div = div-1 then
53                  r_div <= 0;
54                  r_tick <= '1';
55              else
56                  r_div <= r_div + 1;
57                  r_tick <= '0';
58              end if;
```

Snippet 3: `encoder/encoder.vhd`, lines 60–74: edge detection and counter

```
60              if r_tick= '1' then
61                  if ((r_A = '0' and r_A_sync(1) ='1') and r_B_sync(1)= '0') or
62                      ((r_A='1' and r_A_sync(1)='0') and r_B_sync(1)='1') then
63
64                      r_cnt <= r_cnt + 1; --increment
65
66                  elsif ((r_B='0' and r_B_sync(1)='1') and r_A_sync(1)='0') or
67                      ((r_B='1' and r_B_sync(1)='0') and r_A_sync(1)='1') then
68
69                      r_cnt <= r_cnt - 1; --decrement
70
71                  end if;
72
73              r_A <= r_A_sync(1);
74              r_B <= r_B_sync(1);
```

Snippet 4: `encoder/encoder_tb.vhd`, lines 38–51: stimulus and checks

```
38          -- right turn
39          s_A <= '1'; wait for 100 ns;
40          s_B <= '1'; wait for 100 ns;
41          s_A <= '0'; wait for 100 ns;
42          s_B <= '0'; wait for 100 ns;
43          assert s_cnt = to_unsigned(2, 10);
44
45
46          -- left turn: 00 -> 01 -> 11 -> 10 -> 00
47          s_B <= '1'; wait for 100 ns;
48          s_A <= '1'; wait for 100 ns;
49          s_B <= '0'; wait for 100 ns;
```

```
50          s_A <= '0'; wait for 100 ns;
51          assert s_cnt = to_unsigned(0, 10);
```

Snippet 5: hdmi/hdmi_controler.vhd, lines 11–18: 480p timing generics

```
11          -- Timings magic values (480p)
12          h_sync    : positive := 61;
13          h_fp    : positive := 58;
14          h_bp    : positive := 18;
15
16          v_sync    : positive := 5;
17          v_fp    : positive := 30;
18          v_bp    : positive := 9
```

Snippet 6: hdmi/hdmi_controler.vhd, lines 62–80: horizontal counter, hs and active flag

```
62          elsif rising_edge(i_clk) then
63              if (r_h_count = h_total) then
64                  r_h_count <= 0;
65              else
66                  r_h_count <= r_h_count + 1;
67              end if;
68
69              if ((r_h_count >= h_sync) and (r_h_count /= h_total)) then
70                  o_hdmi_hs <= '1';
71              else
72                  o_hdmi_hs <= '0';
73              end if;
74
75              if (r_h_count = h_start) then
76                  r_h_active <= '1';
77              elsif (r_h_count = h_end) then
78                  r_h_active <= '0';
79              end if;
80          end if;
```

Snippet 7: hdmi/hdmi_controler.vhd, lines 123–129: pixel coordinates and address

```
123     -- Generate address
124     o_pixel_en <= '1' when (r_v_active = '1') and (r_h_active = '1') else '0';
125
126     s_x_counter <= (r_h_count - h_start - 1) when (r_h_count > h_start) and (r_h_count <
        h_res + h_start + 1) else 0;
127     s_y_counter <= (r_v_count - v_start - 1) when (r_v_count > v_start) and (r_v_count <
        v_res + v_start + 1) else 0;
128
129     o_pixel_address <= s_x_counter + (s_y_counter * h_res); --r_pixel_counter;
```

Snippet 8: telecran/telecran.vhd, lines 142–144: colour test (in comments)

```
142     --o_hdmi_tx_d(23 downto 16) <= std_logic_vector(to_unsigned(s_x_counter, 8));
143     --o_hdmi_tx_d(15 downto 8)  <= std_logic_vector(to_unsigned(s_y_counter, 8));
144     --o_hdmi_tx_d(7 downto 0)   <= (others => '0');
```

Snippet 9: telecran/telecran.vhd, lines 146–154: moving pixel (in comments)

```
146     -- moving a pixel
147     --process(s_x_counter, s_y_counter, s_left_cnt, s_right_cnt)
148     --begin
```

```
149     --    if (s_x_counter = to_integer(s_left_cnt)) and (s_y_counter = to_integer(
        s_right_cnt)) then
150     --        o_hdmi_tx_d <= x"FFFFFF"; -- white: the cursor is here
151     --    else
152     --        o_hdmi_tx_d <= x"000000"; -- black
153     --    end if;
154     --end process;
```

Snippet 10: `telecran/telecran.vhd`, lines 158–172: memory instance

```
158     -- now for ram
159     dpram0 : entity work.dpram
160         generic map (mem_size => h_res*v_res, data_width => 1)
161         port map(
162             i_clk_a => s_clk_27,
163             i_clk_b =>s_clk_27,
164             i_data_a(0) => s_wr_data,
165             i_addr_a => s_wr_addr,
166             i_we_a => s_wr_we,
167             o_q_a => open,
168             i_data_b => (others => '0'),
169             i_addr_b => s_pixel_address,
170             i_we_b => '0',
171             o_q_b(0) => s_rd_data
172         );
```

Snippet 11: `telecran/telecran.vhd`, lines 174–180: write side of port A and multiplexer

```
174     s_cursor_on_screen <= '1' when (to_integer(s_left_cnt) < h_res) and (to_integer(
        s_right_cnt) < v_res) else '0';
175
176     s_wr_addr <= s_erase_addr when s_erasing = '1' else
177                  to_integer(s_left_cnt) + to_integer(s_right_cnt) * h_res when
        s_cursor_on_screen = '1' else
178                  0;
179     s_wr_data <= '0' when s_erasing = '1' else '1';
180     s_wr_we   <= '1' when (s_erasing = '1') or (s_cursor_on_screen = '1') else '0';
```

Snippet 12: `telecran/telecran.vhd`, line 182: pixel colour

```
182     o_hdmi_tx_d <= x"FFFFFF" when s_rd_data = '1' else x"000000";
```

Snippet 13: `telecran/telecran.vhd`, lines 184–202: erase counter, button synchronizer and select signal

```
184     process(s_clk_27, s_rst_n)
185     begin
186         if s_rst_n = '0' then
187             s_erase_addr <= 0;
188             r_clear_done <= '0';          -- after KEY0 : erase the whole RAM once
189             r_pb_sync    <= "11";
190         elsif rising_edge(s_clk_27) then
191             r_pb_sync <= r_pb_sync(0) & i_left_pb;
192             if s_erase_addr = h_res*v_res-1 then
193                 s_erase_addr <= 0;
194                 r_clear_done <= '1';      -- first full turn done
195             else
196                 s_erase_addr <= s_erase_addr + 1;
197             end if;
198         end if;
```

```
199     end process;
200
201     -- erase while the left button is held ('0' = pressed), or during the first turn after a
          reset
202     s_erasing <= '1' when (r_pb_sync(1) = '0') or (r_clear_done = '0') else '0';
```